

and the timeout parameter is not relevant, at least for now.

FUTEX WAKE: The kernel can wake up a maximum *val* number of processes waiting on this futex. The last three parameters are ignored for this operation. In our experiment, we have used PI-FUTEX, which provides priority inheritance. Functions FUTEX LOCK PI and FUTEX UNLOCK PI are used for locking and unlocking, respectively. The futex value stored at *uaddr* is either zero for unlocked or PID of the owner process. The unlocking of futex allows only a high priority process to wake up, thus avoiding priority inversion.

fu.h: The *fu.h* includes all the header files and functions which are common to *PriInLow.c*, *PriInMed.c* and *PriInHigh.c*. It also includes the *gettime* and *dosomething* functions, which do exactly the same work as that mentioned in the previous section. The *fu.h* includes *struct shm* (shared memory) containing the *futex_add* variable, which is about to be shared between *PriInLow.c* and *PriInHigh.c* for locking purposes. The *FutexLock* and *FutexUnlock* function calls *FUTEX_LOCK_PI* and *FUTEX_UNLOCK_PI* for *Futex_add*, respectively. When futex is locked, then the value of *Futex_add* is equal to the PID of the process that has the lock; else, it will be zero.

fu.c: The *fu.c* creates the shared memory for *Futex_add*. Here, the *ftok* function returns the key depending on the current path files and ID 'b'. The *shmget* creates an identifier associated with the key. The *shmat* attaches the shared memory segment to the address space of the calling process.

PriInLow.c, PriInHigh.c and PriInMed.c: As *PriInLow.c* and *PriInHigh.c* share a futex, before calling *doSomething*, we call the *FutexLock* function and after *doSomething*, the *FutexUnlock* function is called. Whereas in *PriInMed.c*, there is no need to call the *FutexLock* and *FutexUnlock* functions to call the *doSomething* function.

Results and discussions

Single-core operation: We started our discussion with a single-core processor. Three processes with different priorities (say, *Hi*, *Med* and *Lo* processes) are allowed to run on a single-core processor by setting the same CPU affinity. The priorities and policies (*sched FIFO*) of the processes are set before entering into the critical section (CS).

In our experimental setup, we considered the time to execute the CS, which records the start and end time of executing the CS for each process. To execute this CS, *Hi* and *Lo* are sharing resources, whereas *Med* executes a different CS. Since *Lo* and *Hi* share resources, they also share the same lock to access this CS. And since *Med* doesn't share any resources with them, it doesn't need the lock.

When processes *Lo* and *Hi* use a binary semaphore: All the code for *sem.c*, *PriInvLow.c*, *PriInvHigh.c* and *PriInvMed.c* is compiled and their executables are stored as *a.out*, *low*, *high* and *med* respectively. *a.out* is executed first, and then all the remaining run, almost together.

First *Lo* acquires the semaphore and enters into the CS.

During this period, *Hi* and *Med* arrive. As *Hi* needs the semaphore to enter into the CS, it can't start its execution. But *Med* doesn't need the semaphore and also, its priority is greater than *Lo*; hence, it starts its execution pre-empting *Lo*. Thus, *Lo* requires extra time to complete its task, resulting in extra delay in providing resources to the high priority process (*Hi*).

```
root@dhanashree-Inspiron-5559:/media/dhanashree/New Volume/
Study/sem2/ProjectElective/Final Test# ./low
```

```
Low process arrived at : 1522160415
```

```
Press any button to enter into critical section
```

```
start Time: 1522160428
```

```
Low Priority Process started working at : 1522160428
```

```
My Priority is : 20
```

```
Low.Priority Process
```

```
root@dhanashree-Inspiron-5559:/media/dhanashree/New Volume/
Study/sem2/ProjectElective/Final Test# ./high
```

```
High process arrived at : 1522160421
```

```
Press any button to enter into critical section
```

```
High process waiting for lock
```

```
High Priority Process
```

```
root@dhanashree-Inspiron-5559:/media/dhanashree/New Volume/
Study/sem2/ProjectElective/Final Test# ./med
```

```
Med Priority Process arrived at: 1522160425
```

```
Press any button to start work...
```

```
start Time: 1522160431
```

```
Med Priority Process started working at : 1522160431
```

```
My Priority is : 28
```

```
Medium Priority Process
```

From the above code, we can see that *Med* pre-empted and started its work when *Lo* was actually into its critical section.

```
root@dhanashree-Inspiron-5559:/media/dhanashree/New Volume/
Study/sem2/ProjectElective/Final Test# ./low
```

```
Low process arrived at : 1522160415
```

```
Press any button to enter into critical section
```

```
start Time: 1522160428
```

```
Low Priority Process started working at : 1522160428
```

```
My Priority is : 20
```

```
My Priority is : 20
```

```
endTime: 1522160451
```

```
Process completed
```

```
root@dhanashree-Inspiron-5559:/media/dhanashree/New Volume/
Study/sem2/ProjectElective/Final Test#
```